MUTUA ESCLUSIONE

Permette di garantire che, in un determinato momento, un processo ha un accesso esclusivo a delle risorse comuni.
N ≥ 2 processi eseguono in modo continuo due sequenze di istruzioni:
- una sezione non critica (SNC)
- una sezione critica (SC)

In tal modo che la mutua esclusione sia garantita in SC, in ogni istante c'è al massimo un processo in sezione critica.

Proprietà da verificare:
- Mutua esclusione: al massimo un processo in SC ad ogni momento
- Assenza di deadlock: se più processi provano di accedere alla SC nello stesso momento, non devono essere bloccati nel pre-protocollo e almeno un processo deve riuscirci
- Assenza di starvation: se un processo desidera accedere alla sua SC, prima o poi ci riuscirà
- Attesa limitata: se un processo deve attendere per accedere alla sua SC, è possibile limitare il numero di volte in cui dovrà concedere il posto a un altro processo tramite una costante

Attesa limitata → Assenza di starvation → Assenza di deadlock

MUTEX e VARIABILE CONDIZIONALE

Per proteggere l'accesso ai dati condivisi, in C si possono usare dei lock (mutex).
Come funziona:
- un thread che desidera accedere a un dato condiviso richiede il lock
- Se il lock è libero, il thread procede
- Se il lock è preso, il thread rimane bloccato finché il lock non viene liberato
- Quando ha finito di accedere ai dati condivisi, libera il lock

Questi lock sono condivisi fra i threads

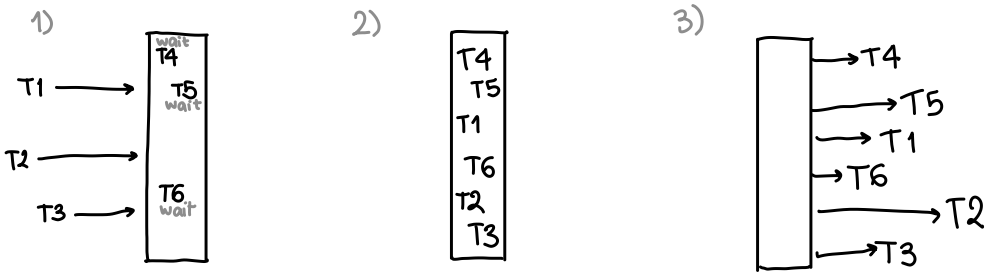
La libreria POSIX ha lock di tipo `pthread_mutex_t lock;`
Per iniziare un lock ci sono due modi:
- `lock = PTHREAD_MUTEX_INITIALIZER` (solo alla dichiarazione)
- `pthread_mutex_init(&lock, NULL)`

Prendere il lock → `int pthread_mutex_lock(pthread_mutex_t *mutex);`
Liberare il lock → `int pthread_mutex_unlock(pthread_mutex_t *mutex);`
} returnano 0 se esito positivo

Distruggere il lock → `pthread_mutex_destroy(&lock)`

BARRIERE

Sono un meccanismo di sincronizzazione utilizzato per esempio nel calcolo parallelo
Rappresenta un punto di sincronizzazione per N thread:
- i thread che arrivano alla barriera aspettano gli altri
- quando tutti gli N thread arrivano alla barriera, allora possono proseguire




```
pthread_barrier_t bar;
pthread_barrier_init(&bar, NULL, 5);
pthread_barrier_wait(&bar);
```

numero di threads che devono arrivare

PROBLEMA DELLA SEZIONE CRITICA:

ALGORITMO DI DEKKER

Un invariante è una proprietà che è vera in tutti gli stati; un invariante induttivo è una proprietà che è vera in uno stato e per tutti gli stati successivi.

L'algoritmo di Dekker verifica:

- gli invarianti (induttivi) successivi
- la mutua esclusione
- l'assenza di deadlock
- l'assenza di starvation sotto le seguenti ipotesi:
  - equità tra processi
  - la SC termina sempre e finisce con l'esecuzione del post-protocollo

ALGORITMO DI PETERSON

L'algoritmo di Peterson verifica:

- la mutua esclusione
- l'assenza di deadlock
- l'assenza di starvation sotto le seguenti ipotesi:
  - equità tra processi
  - la SC termina sempre e finisce con l'esecuzione del post-protocollo

P1	P2	D1	D2	turn
p1	q1	F	F	1
p4	q1	T	F	2
p5	q1	T	F	2
p5	q4	T	T	1
p4	q4	T	T	2

→ P2 è bloccato finchè D1 passa a F oppure turn passa a 2

P2 andrà in sezione critica

```
int turn=1; //variabili condivise
bool D1=false;
bool D2=false;
```

```
Process P1
while(true){
p1: SNC
p2: D1=true;
p3: turn=2;
p4: while(turn!=1 &&D2==true){}
p5:  SC
p6:D1=false;}
```

```
Process P2
while(true){
q1: SNC
q2: D2=true;
q3: turn=1;
q4: while(turn!=2 &&D1==true){}
q5:  SC
q6:D2=false;}
```



Starvation dove P1 rimane bloccato in p4 while(...){}

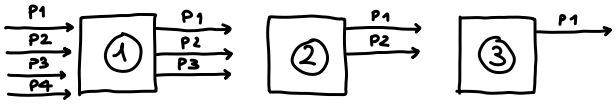
ogni volta che P1 fa p4, abbiamo per forza turn=2 e D2=True

PASSARE DA 2 A N>2 PROCESSI

Ci sono diversi algoritmi:

- **Filter lock**

4 processi



Ci sono N-1 level da "attraversare".
Per ogni level:

- tra i processi che provano a passare, almeno uno riesce
- se ci sono più processi che provano, almeno uno viene bloccato.

level[id]: J → processo P[id] prova a passare la "sala d'attesa" J
victim[J]: id → il processo che verrà bloccato nella sala J è id (se più processi provano a passare)

$num_{,J} \leq n-J \quad \forall J \in [0, n-1]$

Supponiamo che $num_{\geq J-1} \leq n-(J-1) \leq n-J+1$] per $J>0$

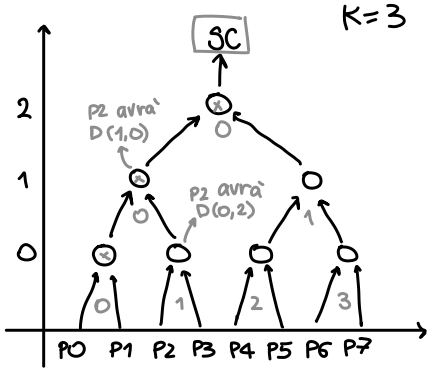
↳ Ipotesi di induzione

→ per forza ho $num_{\geq J} \leq n \cdot J+1$

↳ $num_{\geq J} \leq n \cdot J$

Assenza di starvation implica l'assenza di deadlock!!!
MA assenza di deadlock non implica l'assenza di starvation.

• il torneo



$K=3$ Una partita è identificata dalla coppia (le, k)

↳ turn

$D(le, 2k)$ } corrispettivi di D1 e D2 nell'algoritmo di Peterson
$D(le, 2k+1)$ }

$le \in \{0, \dots, \log(n)-1\}$, $k \in \{0, \dots, 2^{\log(n)-1-le}-1$

↳ giro corrente ↳ numero della partita nel giro l

$id \in \{1,2\}$ → indica se il processo è a sinistra (1) o a destra (2)

$n=2^k$ processi, torneo con eliminazione in $\log(n)=k$ giri. Dopo il primo giro sono eliminati $n/2$ processi, dopo il secondo giro, sono eliminati $n/4$ processi, ecc.
I giri sono numerati da 0 a $\log(n)-1$

• Algoritmo del fornaio (bakery algorithm)

Algoritmo di mutua esclusione per $N \geq 2$ processi.
Usa delle variabili proprie (multiple readers/single writer): ogni processo ha le sue variabili che possono essere lette ma non modificate dagli altri processi

Un nuovo 'cliente' che vuole avere accesso alle risorse prende un numero superiore ai numeri dei clienti già in attesa. Il cliente prioritario è quello che ha il numero più piccolo.